Laboratório Funcional · Aula 12

Gabriel Campos Feiten · RA 24047639-2 · Turma A
Gerado em 16/10/2026, 19:48:27 · tempo registrado no cronômetro: 47:09

Preparação: Programar com funções matemáticas

def area(r): return 3.14159 * r * r

Pura

def agora(): return datetime.now()

Impura

def total(lista): return sum(lista)

Pura

def registra(msg): print(msg)

Impura

def sorteia(): return random.randint(1, 6)

Impura

def poe(lista, x): lista.append(x); return lista

Impura

0.1 Escolha duas funções que você marcou como impuras e explique, para cada uma, o que a impede de ser pura.

Na registra(msg) ela é impura porque tem efeito colateral, que é escrever no terminal. Já a sorteia é porque ela não depende só dos parâmetros, ela vai dar um resultado aleatório a cada chamada

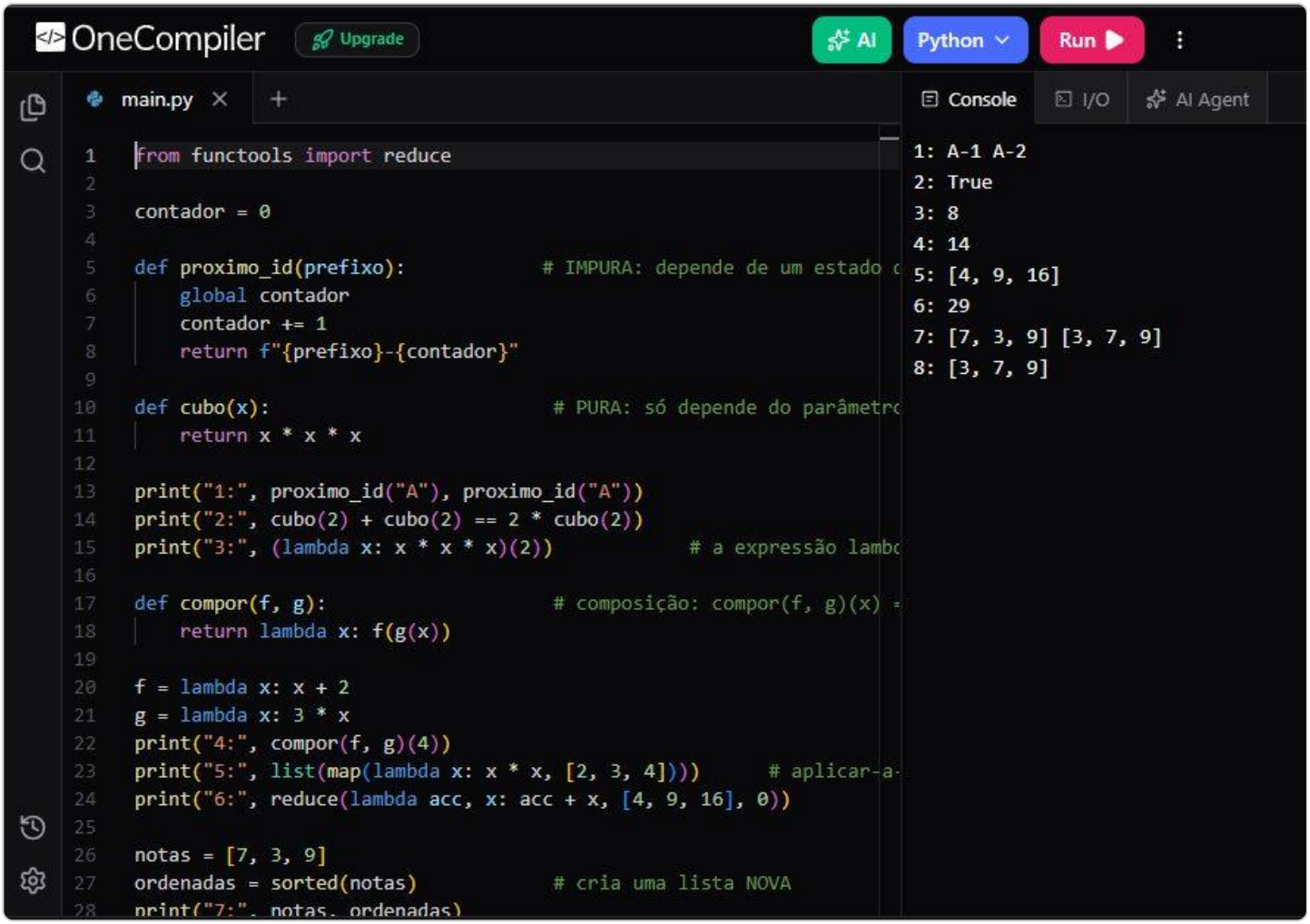
Prática 1: Funções puras e formas funcionais

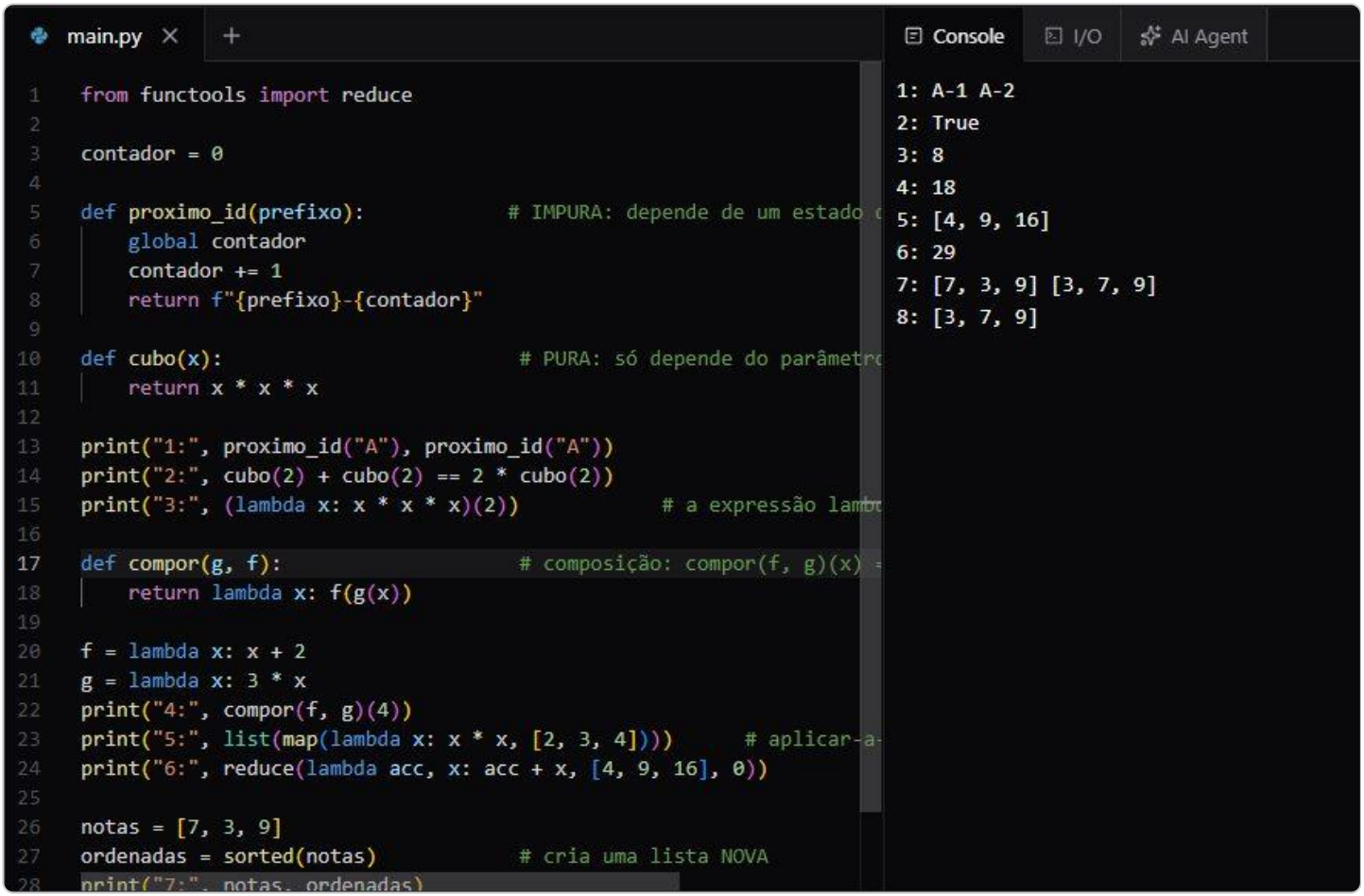
Previsão: O que as linhas 1, 4 e 7 vão imprimir?

Linha 1: A-1, A-2.
Linha 4: 14
Linha 7: [7, 3, 9], [3, 7, 9]

Saídas (original e com compor(g, f))

não respondido


OneCompiler
Upgrade
AI
Python
Run
main.py
Console
I/O
AI Agent
from functools import reduce
contador = 0
def proximo_id(prefixo): # IMPURA: depende de um estado
global contador
contador += 1
return f"{prefixo}-{contador}"
def cubo(x): # PURA: só depende do parâmetro
return x * x * x
print("1:", proximo_id("A"), proximo_id("A"))
print("2:", cubo(2) + cubo(2) == 2 * cubo(2))
print("3:", (lambda x: x * x * x)(2)) # a expressão lambda
def compor(f, g): # composição: compor(f, g)(x)
return lambda x: f(g(x))
f = lambda x: x + 2
g = lambda x: 3 * x
print("4:", compor(f, g)(4))
print("5:", list(map(lambda x: x * x, [2, 3, 4]))) # aplicar-a
print("6:", reduce(lambda acc, x: acc + x, [4, 9, 16], 0))
notas = [7, 3, 9]
ordenadas = sorted(notas) # cria uma lista NOVA
print("7:", notas, ordenadas)
1: A-1 A-2
2: True
3: 8
4: 14
5: [4, 9, 16]
6: 29
7: [7, 3, 9] [3, 7, 9]
8: [3, 7, 9]


main.py
Console
I/O
AI Agent
from functools import reduce
contador = 0
def proximo_id(prefixo): # IMPURA: depende de um estado
global contador
contador += 1
return f"{prefixo}-{contador}"
def cubo(x): # PURA: só depende do parâmetro
return x * x * x
print("1:", proximo_id("A"), proximo_id("A"))
print("2:", cubo(2) + cubo(2) == 2 * cubo(2))
print("3:", (lambda x: x * x * x)(2)) # a expressão lambda
def compor(g, f): # composição: compor(f, g)(x)
return lambda x: f(g(x))
f = lambda x: x + 2
g = lambda x: 3 * x
print("4:", compor(f, g)(4))
print("5:", list(map(lambda x: x * x, [2, 3, 4]))) # aplicar-a
print("6:", reduce(lambda acc, x: acc + x, [4, 9, 16], 0))
notas = [7, 3, 9]
ordenadas = sorted(notas) # cria uma lista NOVA
print("7:", notas, ordenadas)
1: A-1 A-2
2: True
3: 8
4: 18
5: [4, 9, 16]
6: 29
7: [7, 3, 9] [3, 7, 9]
8: [3, 7, 9]

1.1 Por que proximo_id("A") devolve valores diferentes para a mesma entrada? Por que isso dificulta os testes (p. 637)?

Porque a função tem o efeito colateral de alterar uma variável durante sua execução. Dificulta os testes porque não dará para realizar testes repetíveis.

1.2 compor(f, g)(4) e compor(g, f)(4) deram valores diferentes. Calcule os dois à mão e explique a diferença.

porque primeiro multiplica por 3 e depois soma dois (4*3+2) que da 14, e na segunda ela soma (2+4) e depois multiplica por 3, dando 18.

1.3 Qual é a diferença entre sorted(notas) e notas.sort()? Qual dos dois segue o estilo funcional, e por quê?

sorted(notas) cria um vetor ordenado novo e deixa o original intacto, já o notas.sort() ordena o vetor original, modificando-o.

Prática 2: Scheme: tudo é lista

Previsão: (car '(A B C))

A

Previsão: (cdr '(A B C))

(B C)

Previsão: (cons '(A B) '(C D))

((A B) C D)

Saídas (original e sem o apóstrofo)

não respondido

main.scm

```
; Prática 2 - Scheme: o essencial (Sebesta 15.5, p. 641-650)
; Cole em https://onecompiler.com/scheme
(define (mostra x) (display x) (newline))

(mostra (+ 3 7 10 2))              ; notação prefixa: o operador
(mostra (* 2 (+ 3 4)))

(define (quadrado x) (* x x))      ; definir uma função
(mostra (quadrado 5))
(mostra ((lambda (x) (* x x x)) 2)) ; uma função sem nome, aplic

(define (sinal n)
  (cond ((> n 0) 'positivo)
        ((< n 0) 'negativo)
        (else 'zero)))
(mostra (sinal -7))

(mostra (car '(A B C)))            ; o primeiro elemento
(mostra (cdr '(A B C)))            ; o resto da lista
(mostra (cons 'A '(B C)))          ; constrói uma lista
(mostra (cons '(A B) '(C D)))
(mostra (car (cdr '(A B C))))
```

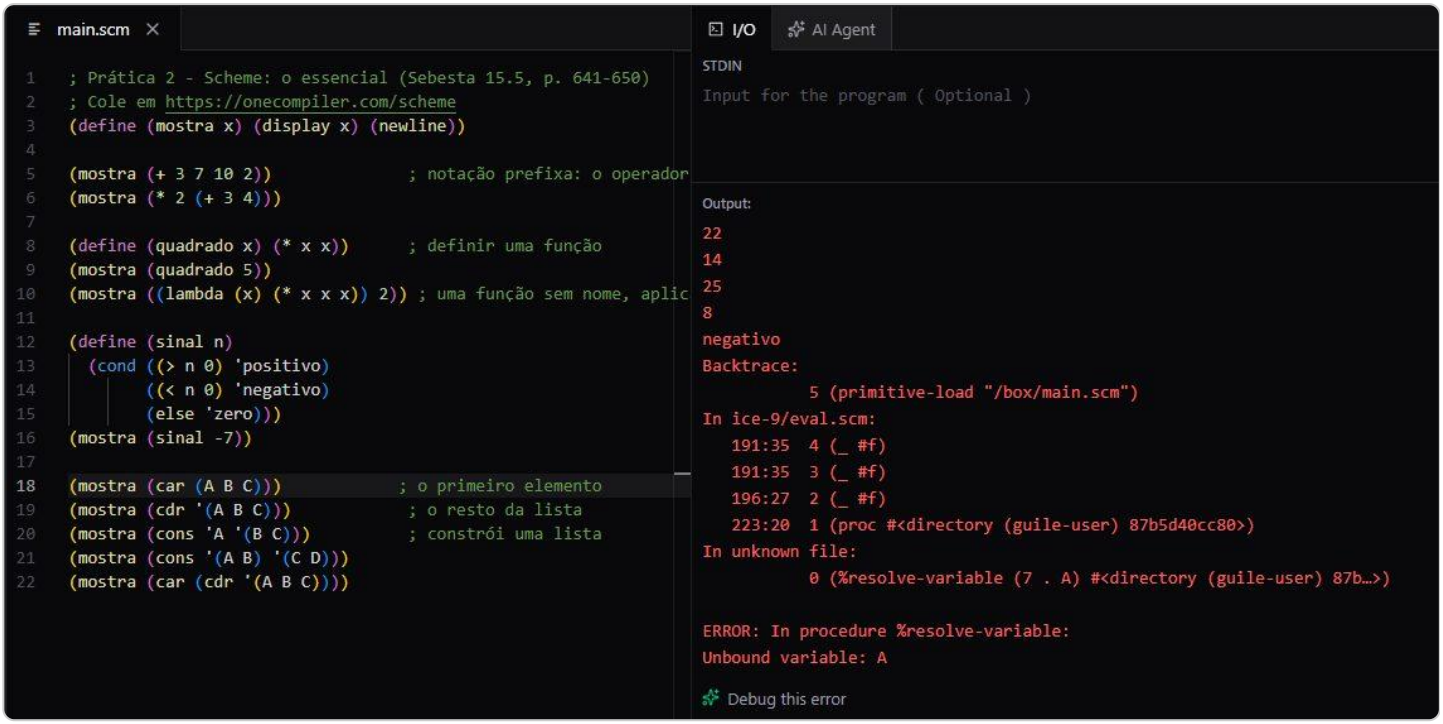
I/O | AI Agent

STDIN

Input for the program (Option

Output:

```
22
14
25
8
negativo
A
(B C)
(A B C)
((A B) C D)
B
```



2.1 Para que serve o apóstrofo (')? O que o Scheme tentou fazer com (A B C) quando ele não estava lá?

O apóstrofo (') indica que a expressão deve ser tratada como dado, sem ser executada. Sem ele, o Scheme tentaria interpretar (A B C) como uma chamada de função, usando A como procedimento e B e C como argumentos, causando um erro se A não estiver definido como procedimento.

2.2 Escreva uma expressão que devolva o terceiro elemento de '(A B C) usando só car e cdr. Teste no OneCompiler e cole aqui a expressão e o resultado.

```
(car (cdr (cdr '(A B C))))
```

Prática 3: Recursão e recursão em cauda

O que vi no passo a passo

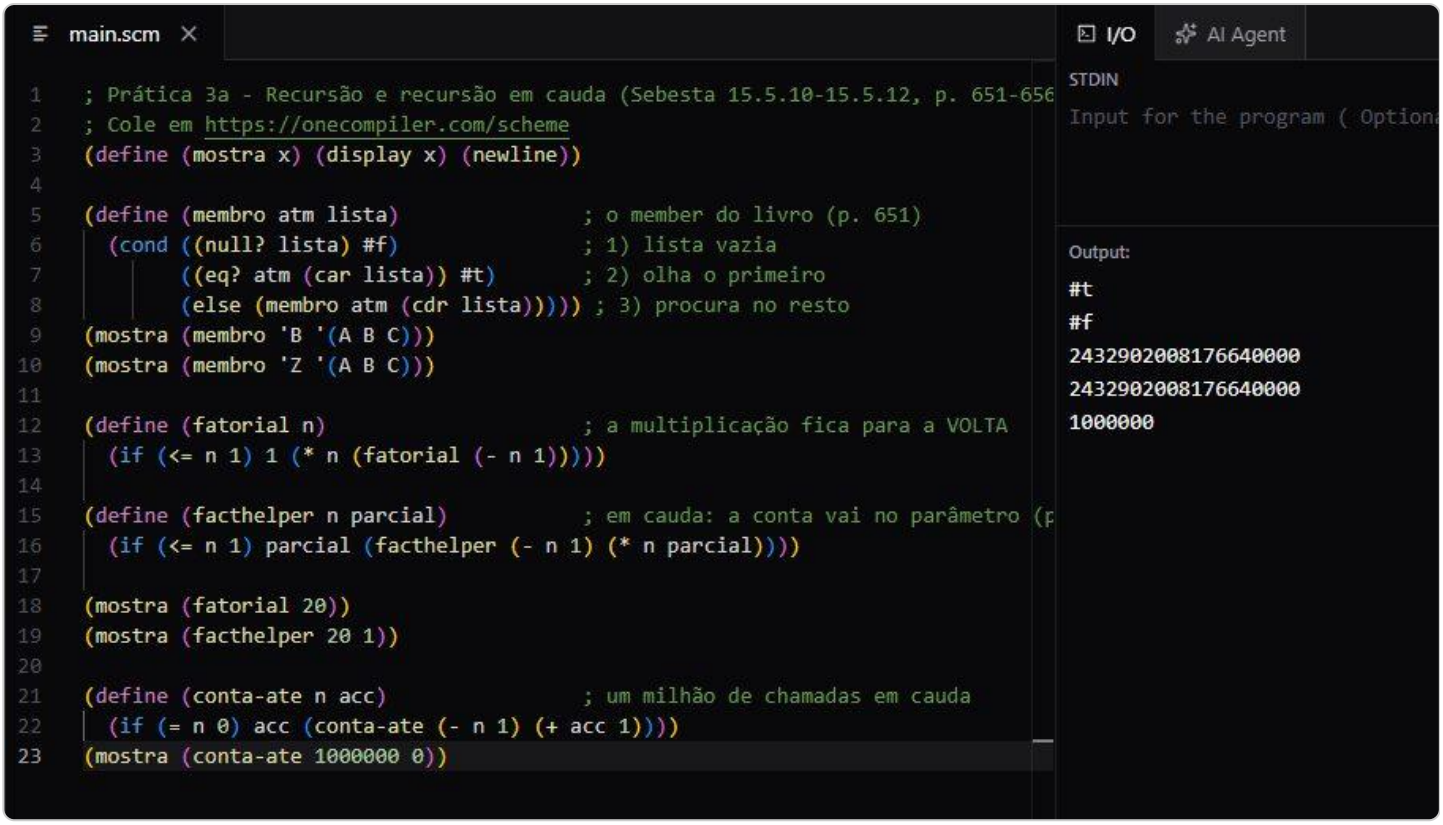
n = 3: comum em 6 passos, até 2 multiplicações esperando (expressão de até 24 caracteres); em cauda em 5 passos, nada esperando (até 16 caracteres).
n = 4: comum em 8 passos, até 3 multiplicações esperando (expressão de até 30 caracteres); em cauda em 6 passos, nada esperando (até 17 caracteres).
n = 5: comum em 10 passos, até 4 multiplicações esperando (expressão de até 36 caracteres); em cauda em 7 passos, nada esperando (até 18 caracteres).
n = 6: comum em 12 passos, até 5 multiplicações esperando (expressão de até 42 caracteres); em cauda em 8 passos, nada esperando (até 18 caracteres).

Previsão: O programa em Scheme chama conta-ate recursivamente um milhão de vezes. Vai funcionar? E a mesma função em Python?

Vão funcionar

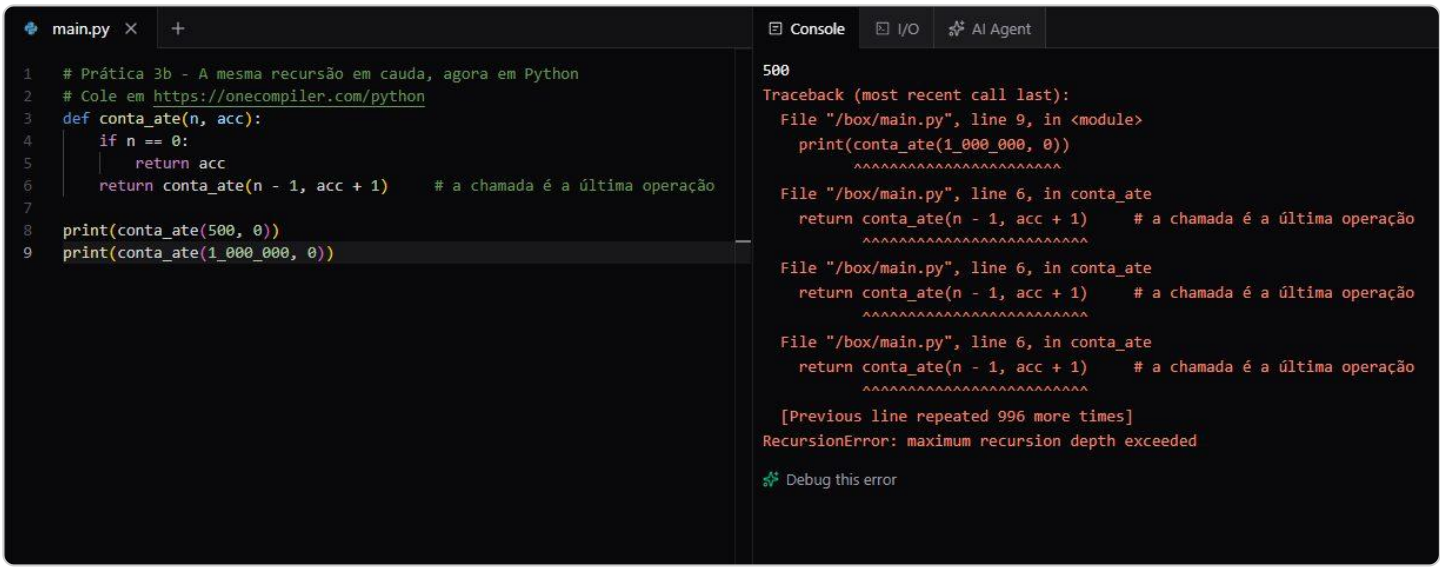
Saída do Scheme

não respondido



Saída do Python (as últimas linhas bastam)

não respondido



3.1 No passo a passo, o que cresce na recursão comum e não cresce na recursão em cauda? Por quê?

Na recursão comum, cresce a quantidade de operações pendentes na pilha, pois as multiplicações precisam esperar as chamadas recursivas terminarem. Na recursão em cauda, isso não acontece porque o cálculo é realizado antes de cada nova chamada, acumulando o resultado no parâmetro.

3.2 Por que o Scheme fez um milhão de chamadas e o Python parou com RecursionError? (Dica: releia o quadro sobre a otimização da chamada em cauda e a mensagem de erro do Python. O livro fala do Scheme na p. 656.)

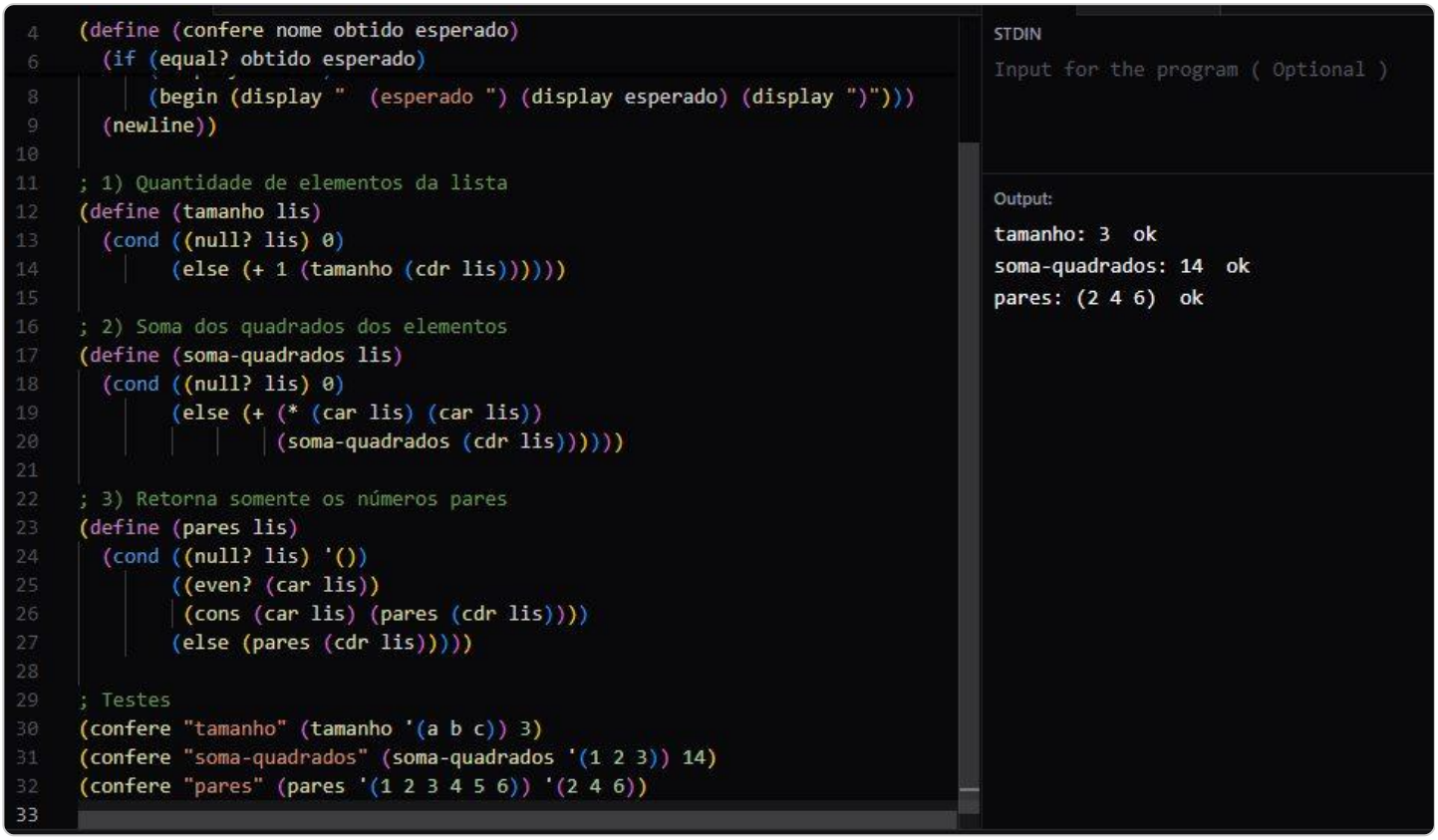
O Scheme consegue executar um milhão de chamadas porque implementa otimização de recursão em cauda, reutilizando o espaço de execução e evitando acumular chamadas na pilha. Já o Python não realiza essa

otimização, mesmo quando a chamada recursiva é a última operação, atingindo o limite de recursão e gerando RecursionError

Prática 4: Mão na massa: três funções sobre listas

Saída final (com os três ok)

não respondido



4.1 Cole aqui as suas três funções (tamanho, soma-quadrados e pares).

```
(define (tamanho lis)
  (cond ((null? lis) 0)
        (else (+ 1 (tamanho (cdr lis))))))

(define (soma-quadrados lis)
  (cond ((null? lis) 0)
        (else (+ (* (car lis) (car lis))
                 (soma-quadrados (cdr lis))))))

(define (pares lis)
  (cond ((null? lis) '())
        ((even? (car lis))
         (cons (car lis) (pares (cdr lis))))
        (else (pares (cdr lis)))))
```

4.2 Qual foi a parte mais difícil? Onde você usou null?, car e cdr em cada função?

A parte mais difícil foi entender como a recursão percorre a lista. Usei null? para verificar se a lista estava vazia, car para acessar o primeiro elemento e cdr para continuar a operação com o restante da lista.

Prática 5: Haskell: padrões, compreensões e preguiça

Previsão: squares é uma lista infinita. O que você acha que take 5 squares imprime? E primeiro 42 (div 1 0)?

[0,1,4,9,16], 42

Saídas (original e com member 17)

não respondido

Main.hs

```haskell
-- Prática 5 - Haskell: padrões, compreensões e avaliação preguiçosa (Sebesta 15.8, p
-- Cole em https://onecompiler.com/haskell
fact 0 = 1                                  -- casamento de padrões (p. 667)
fact n = n * fact (n - 1)

factors n = [i | i <- [1 .. n `div` 2], n `mod` i == 0]   -- compreensão de lista (p.

sort []     = []                            -- quicksort (p. 669, corrigido)
sort (h:t) = sort [b | b <- t, b <= h] ++ [h] ++ sort [b | b <- t, b > h]

squares = [n * n | n <- [0 ..]]             -- uma lista INFINITA (p. 670)

member b []     = False                     -- o member do livro (p. 670)
member b (a:x) = (a == b) || member b x

primeiro x _ = x

main :: IO ()
main = do
  print (fact 20)
  print (factors 28)
  print (sort [7, 2, 9, 4, 2, 8])
  print (take 5 squares)
  print (takeWhile (< 50) squares)
  print (member 16 squares)
  print (primeiro 42 (div 1 0))
  -- print (member 17 squares)              -- tire o "--" do começo desta linha
```

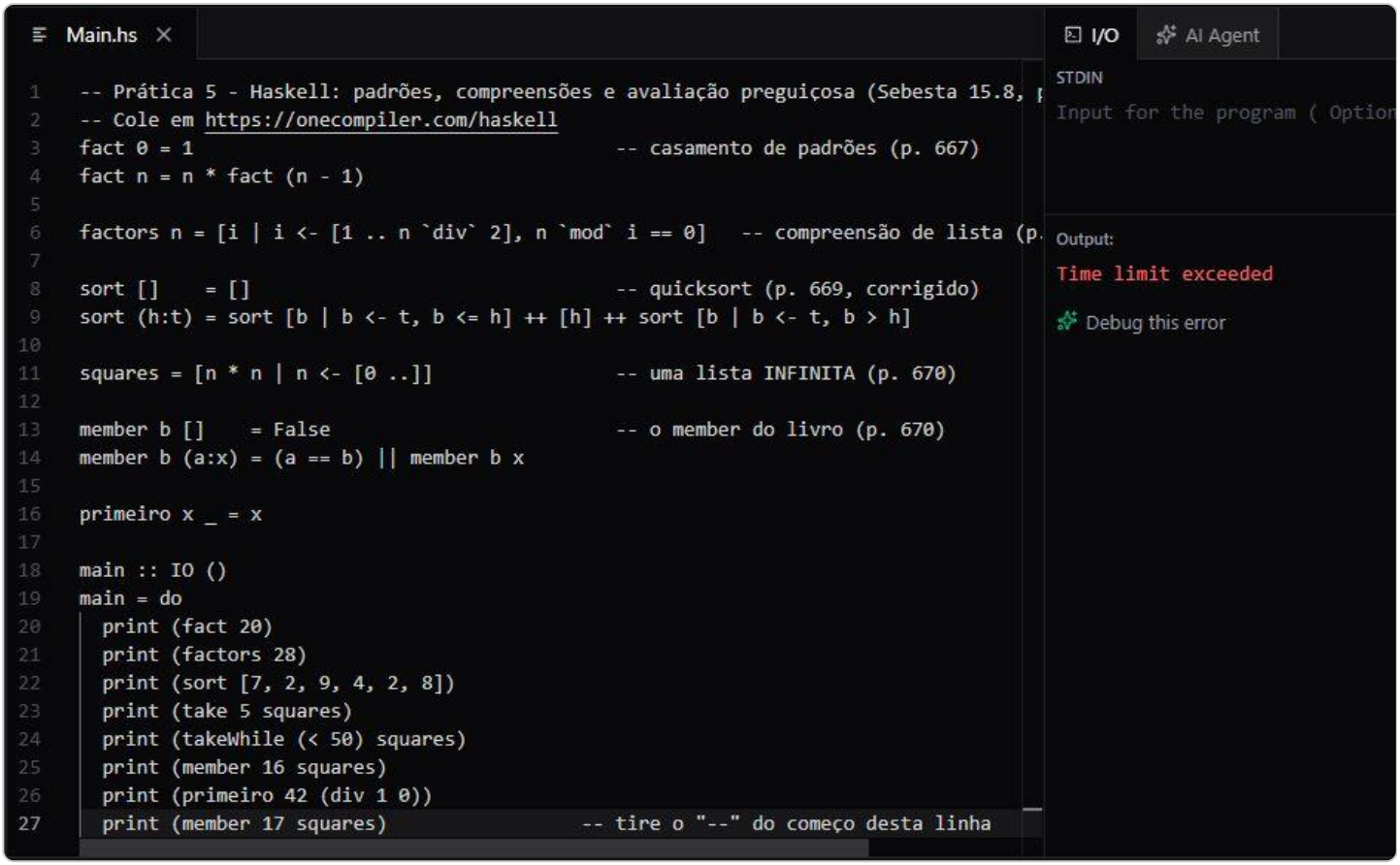
I/O | AI Agent

STDIN

Input for the program (Opti

Output:

```
2432902008176640000
[1,2,4,7,14]
[2,2,4,7,8,9]
[0,1,4,9,16]
[0,1,4,9,16,25,36,49]
True
42
```



5.1 Como o Haskell consegue imprimir parte de uma lista infinita? E por que primeiro 42 (div 1 0) não deu erro de divisão por zero?

O Haskell utiliza avaliação preguiçosa, calculando apenas os elementos necessários da lista infinita. No caso de primeiro 42 (div 1 0), o segundo argumento não é avaliado, pois a função retorna apenas o primeiro argumento, evitando o erro de divisão por zero.

5.2 O que aconteceu com member 17 squares? Por que ele nunca termina, enquanto member 16 squares terminou? Como você corrigiria? (Dica: squares está em ordem crescente. Quando já dá para desistir da busca? O livro mostra uma solução na p. 670.)

member 17 squares nunca termina porque percorre a lista infinita procurando 17, que não é um quadrado perfeito. Já member 16 squares termina porque encontra o valor na lista. Eu corrigiria verificando se o elemento atual é maior que o valor procurado; como a lista está em ordem crescente, nesse caso já podemos retornar False e encerrar a busca.

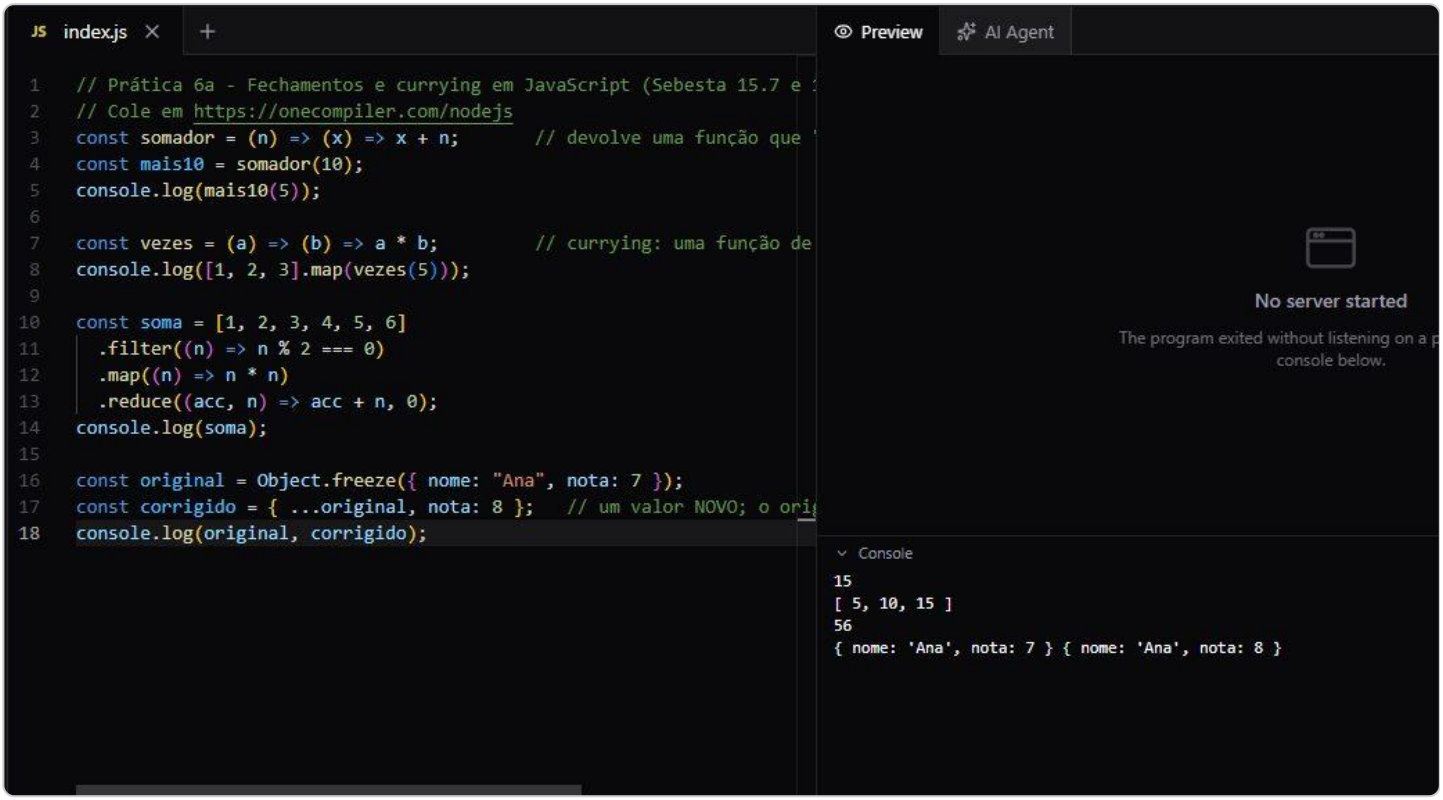
Prática 6: Você já usa funcional

Previsão: O que mais10(5) e [1, 2, 3].map(vezes(5)) vão imprimir?

15 e [5, 10, 15]

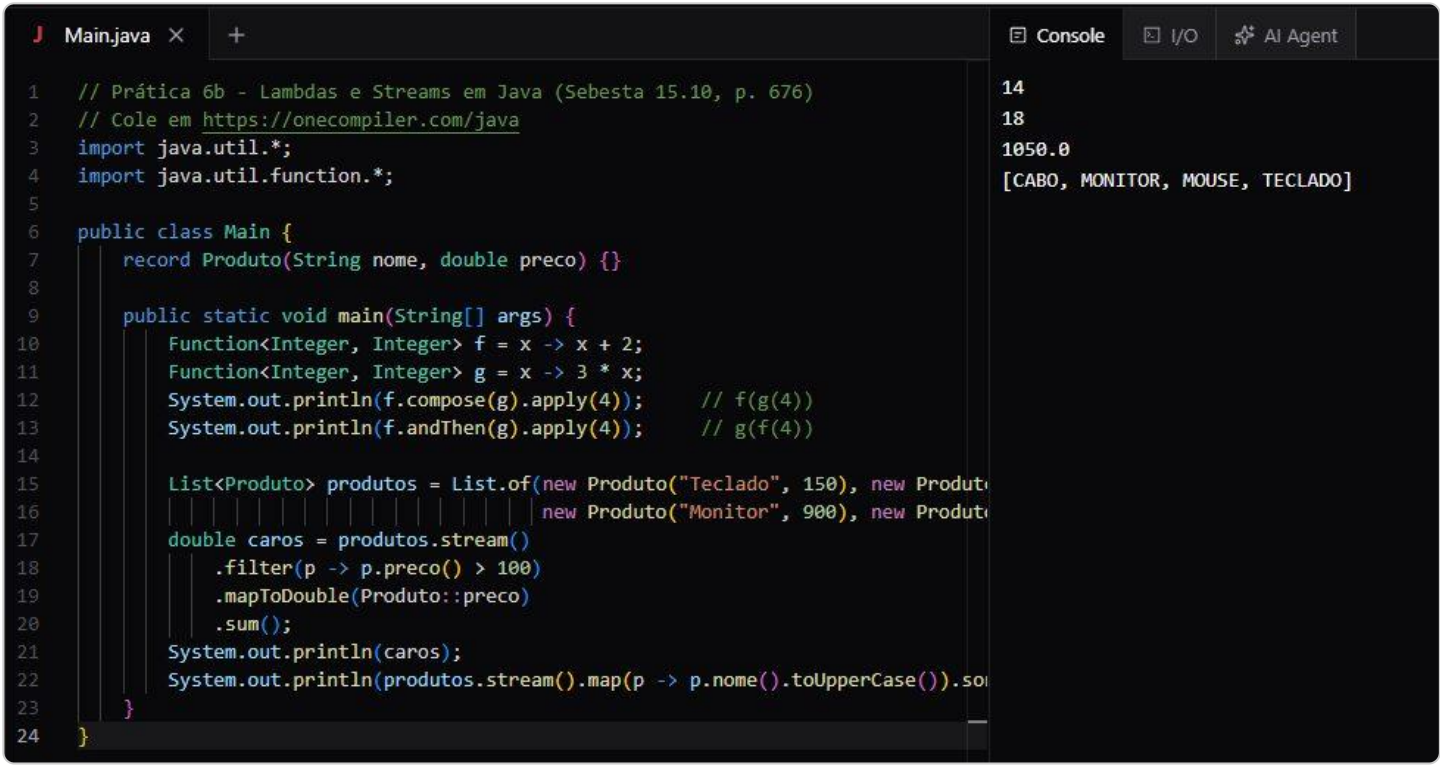
Saída do JavaScript

não respondido



Saída do Java

não respondido



6.1 somador(10) já terminou de executar quando mais10(5) é chamado. Como a função devolvida ainda "lembra" o 10?

Isso acontece por causa do fechamento (closure): a função devolvida mantém acesso ao ambiente em que foi criada, preservando o valor de n = 10, mesmo depois que somador(10) termina de executar.

6.2 Por que f.compose(g).apply(4) e f.andThen(g).apply(4) deram resultados diferentes? Qual deles corresponde ao compor(f, g) da Prática 1?

Os resultados são diferentes porque a ordem de execução das funções muda. f.compose(g).apply(4) executa primeiro g(4) = 12 e depois f(12) = 14. Já f.andThen(g).apply(4) executa primeiro f(4) = 6 e depois g(6) = 18. f.compose(g) corresponde ao compor(f, g) da Prática 1, pois executa f(g(x))

Prática 7: O laço vira pipeline

7.1 Primeiro exercício escolhido (1, 2 ou 3): cole o seu código reescrito.

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        List<Integer> nums = List.of(3, 8, 5, 12, 7, 6);

        int soma = nums.stream()
            .filter(n → n % 2 == 0)
            .mapToInt(n → n * n)
            .sum();

        System.out.println(soma);
    }
}
```

7.2 Segundo exercício escolhido: cole o seu código reescrito.

```
const alunos = [
  { nome: "Ana", nota: 8.5 },
  { nome: "Bruno", nota: 5.0 },
  { nome: "Carla", nota: 7.0 },
  { nome: "Davi", nota: 9.5 },
];

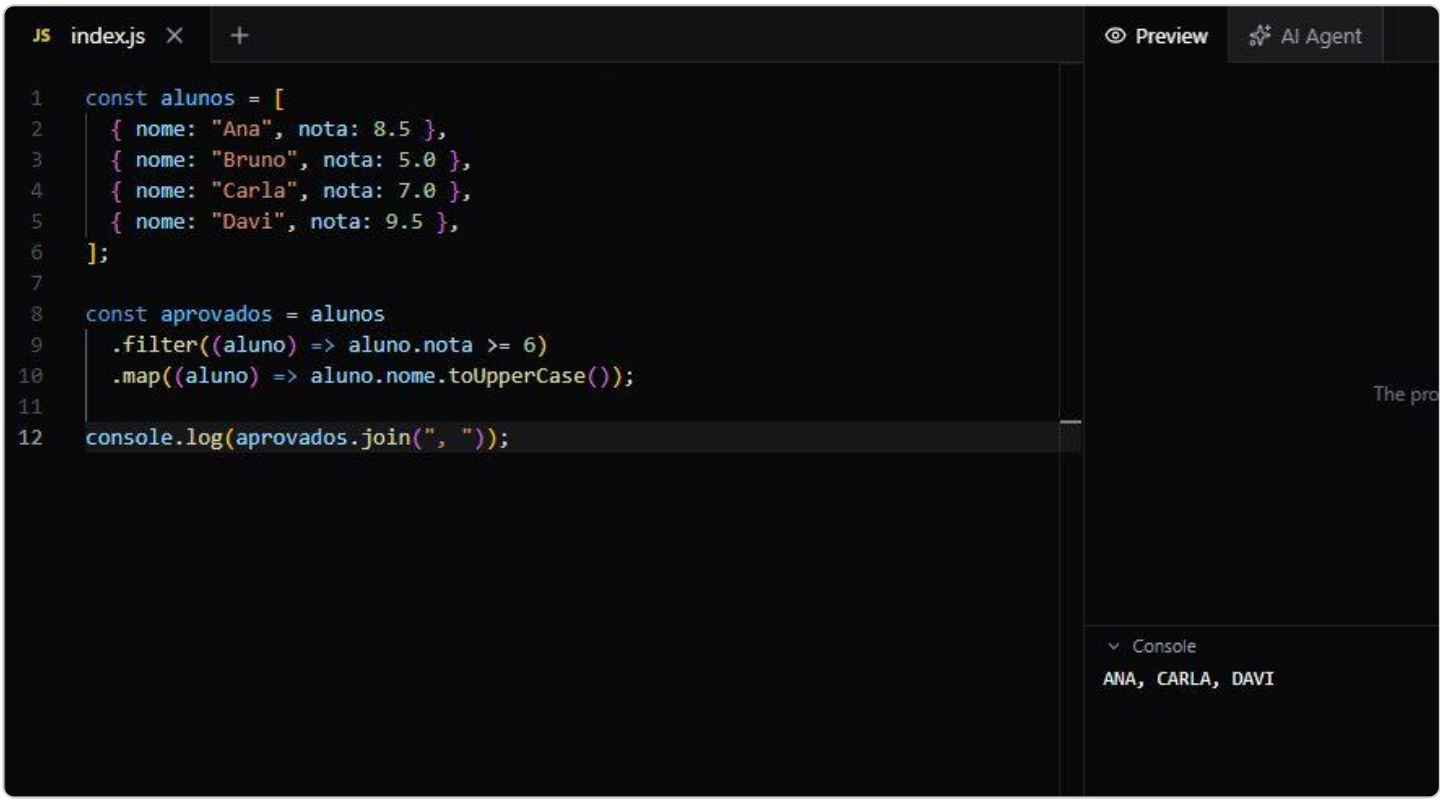
const aprovados = alunos
  .filter((aluno) ⇒ aluno.nota ⩾ 6)
  .map((aluno) ⇒ aluno.nome.toUpperCase());

console.log(aprovados.join(", "));
```

Saídas das suas versões

não respondido





7.3 Qual versão é mais fácil de ler: a com laço ou a funcional? E qual é mais fácil de depurar (por exemplo, para ver um valor intermediário)?

A versão funcional é mais fácil de ler, pois organiza as operações em etapas como filter e map. Já a versão com laço pode ser mais fácil de depurar, pois permite acompanhar cada iteração e verificar os valores intermediários com mais facilidade.

Fechamento: O que fica do paradigma funcional

Uma função pura pode ler uma variável global que muda.

Falso

Em Scheme, a recursão em cauda não faz a pilha crescer.

Verdadeiro

O Python otimiza a recursão em cauda.

Falso

Em Haskell, [1 ..] só é calculada à medida que é usada.

Verdadeiro

filter, map e reduce dispensam laço e variável acumuladora.

Verdadeiro

Currying transforma uma função de vários parâmetros numa cadeia de funções de um parâmetro.

Verdadeiro

8.1 Qual versão é mais fácil de entender, e por quê? O livro estima que um programa funcional costuma levar cerca de 2 vezes mais tempo para executar (p. 678). Em que tipo de sistema você aceitaria esse custo, e em qual não?

A versão em Haskell é mais fácil de entender porque expressa diretamente as operações de gerar a lista, elevar os valores ao cubo e somá-los, sem precisar controlar um laço. Eu aceitaria o custo de execução em sistemas de análise de dados, mas não em sistemas de tempo real, nos quais o desempenho e a rapidez são essenciais.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

O que mais me surpreendeu foi perceber que a programação funcional pode tornar o código mais simples e legível, facilitar a manutenção e reduzir efeitos colaterais.

8.3 Qual é o erro de impressão, e como corrigir? O que a mensagem do compilador diz sobre o tipo que ele inferiu?

O erro está em b <- h, que deveria ser b <= h, pois a intenção é selecionar os elementos menores ou iguais ao pivô h. O compilador identifica um erro de tipo, pois h é um número, mas foi usado como se fosse uma lista para percorrer.